

Walmart BUSINESS CASE STUDY

Customer Purchase Behaviour & Central Limit Theorem

By - Shivaling Battarki

Email: shivalingb09@gmail.com

About Walmart

Walmart is an American multinational retail corporation that operates a chain of supercenters, discount departmental stores, and grocery stores from the United States. Walmart has more than 100 million customers worldwide.

Business Problem

The Management team at Walmart Inc. wants to analyze the customer purchase behavior (specifically, purchase amount) against the customer's gender and the various other factors to help the business make better decisions. They want to understand if the spending habits differ between male and female customers: Do women spend more on Black Friday than men? (Assume 50 million customers are male and 50 million are female).

Dataset

The company collected the transactional data of customers who purchased products from the Walmart Stores during Black Friday. The dataset has the following features:

Dataset link: [Walmart_data.csv](#)

User_ID:	User ID
Product_ID:	Product ID
Gender:	Sex of User
Age:	Age in bins
Occupation:	Occupation(Masked)
City_Category:	Category of the City (A,B,C)
StayInCurrentCityYears:	Number of years stay in current city
Marital_Status:	Marital Status
ProductCategory:	Product Category (Masked)
Purchase:	Purchase Amount

Google Colab Notebook Link:

https://colab.research.google.com/drive/1ZR18Qc0kWvfEN5gZNfYX24BGikaUsR_q?usp=sharing

Project Overview

This case study analyzes Walmart Black Friday transactional data to understand customer purchase behaviour and identify whether spending differs across gender, marital status, and age life stages.

Key Areas Covered

- Dataset structure, characteristics, categorical attributes, and statistical summary.
- Non-graphical and visual analysis of purchase behaviour and customer demographics.
- Missing-value and outlier detection.
- Comparison of male and female purchase behaviour and product-category mix.
- Central Limit Theorem and sampling distributions for different sample sizes.
- 90%, 95%, and 99% confidence intervals for population average spending.
- Comparison of spending between married and unmarried customers.
- Age-based analysis using the life stages: 0–17, 18–25, 26–35, 36–50, and 51+.
- Business insights, population-level interpretation, and actionable recommendations.

Based on the completed Jupyter / Google Colab analysis submitted in the following pages.

Walmart Business Case Study: Customer Purchase Behavior & Central Limit Theorem Analysis

Executive Summary & Business Context

Walmart Inc. is an American multinational retail corporation operating a chain of hypermarkets, discount departmental stores, and grocery stores. With over 100 million customers worldwide, understanding consumer spending dynamics during peak promotional events—especially **Black Friday**—is paramount for strategic decision-making, inventory allocation, pricing, and personalized marketing.

Core Business Problem

The Management team at Walmart Inc. wants to analyze customer purchase behavior (specifically, **purchase amount**) against customer demographics (**Gender, Age, Marital Status, City Category, Stay in City, and Occupation**) to answer fundamental business questions:

1. **Do women spend more on Black Friday than men?** (Assuming a total population of **50 million male** and **50 million female** customers).
2. **What are the Confidence Intervals for average male and female spending?** Do they overlap, and what are the strategic implications for Walmart?
3. **How does spending vary across Marital Status (Married vs Unmarried)?**
4. **How does spending evolve across Age Life Stages (0-17 , 18-25 , 26-35 , 36-50 , 51+)?**
5. **How can the Central Limit Theorem (CLT) be leveraged to make reliable population-level inferences from transactional sample data?**

```
1 # Import essential data analysis and statistical libraries
2 import os
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 import scipy.stats as stats
7 import seaborn as sns
8 import warnings
9 warnings.filterwarnings("ignore")
10
11 # Configure aesthetic visual styling
12 sns.set_theme(style="whitegrid", palette="deep")
13 pd.set_option('display.max_columns', None)
14 pd.set_option('display.float_format', lambda x: '%.3f' % x)
15 plt.rcParams.update({
16     'font.size': 10,          # General/default font size
17     'axes.titlesize': 10,     # Title size
18     'axes.labelsize': 10,     # X and Y axis label size
19     'xtick.labelsize': 10,    # X tick label size
20     'ytick.labelsize': 10,    # Y tick label size
21     'legend.fontsize': 10     # Legend font size
22 })
23
24
25 print("Libraries successfully loaded and configured.")
```

Libraries successfully loaded and configured.

```
1 # Import the Walmart Dataset
2 !wget 1TedL3cSU7eVI01otGj4sCLO28zeEwVDp -O walmart.csv
```

```
Downloading...
From: https://drive.google.com/uc?id=1TedL3cSU7eVI01otGj4sCLO28zeEwVDp
To: /content/walmart.csv
100% 23.0M/23.0M [00:00<00:00, 250MB/s]
```

```
1 # Load the Walmart Dataset
2 walmart = pd.read_csv("walmart.csv")
3
4 print(f"Dataset Dimensions: {walmart.shape[0]} rows and {walmart.shape[1]} columns\n")
5 walmart.head()
```

Dataset Dimensions: 550068 rows and 10 columns

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
0	1000001	P00069042	F	0-17	10	A	2	0	3	8370
1	1000001	P00248942	F	0-17	10	A	2	0	1	15200
2	1000001	P00087842	F	0-17	10	A	2	0	12	1422
3	1000001	P00005110	F	0-17	10	A	2	0	10	1057

1. Defining Problem Statement & Analyzing Basic Metrics

1 # Characteristics of the dataset

1 walmart.columns

```
Index(['User_ID', 'Product_ID', 'Gender', 'Age', 'Occupation', 'City_Category',  
      'Stay_In_Current_City_Years', 'Marital_Status', 'Product_Category',  
      'Purchase'],  
      dtype='object')
```

1 walmart.info()

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 550068 entries, 0 to 550067  
Data columns (total 10 columns):  
#   Column                Non-Null Count  Dtype  
---  ----  
0   User_ID                550068 non-null  int64  
1   Product_ID             550068 non-null  object  
2   Gender                 550068 non-null  object  
3   Age                   550068 non-null  object  
4   Occupation             550068 non-null  int64  
5   City_Category          550068 non-null  object  
6   Stay_In_Current_City_Years  550068 non-null  object  
7   Marital_Status         550068 non-null  int64  
8   Product_Category       550068 non-null  int64  
9   Purchase               550068 non-null  int64  
dtypes: int64(5), object(5)  
memory usage: 42.0+ MB
```

```
1 # Checking for any Duplicate records  
2 # To check for completely identical rows across all columns:  
3 total_rows = len(walmart)  
4 duplicate_rows = walmart.duplicated().sum()  
5 duplicate_percent = (duplicate_rows / total_rows) * 100  
6  
7 print(f"Duplicate Rows: {duplicate_rows} ({duplicate_percent:.2f}%")
```

Duplicate Rows: 0 (0.00%)

```
1 # Missing Value, NULL audit  
2 null_audit = pd.DataFrame({  
3     'Null Count': walmart.isnull().sum(),  
4     'Null Percentage (%)': (walmart.isnull().sum() / len(walmart)) * 100  
5 })  
6 display(null_audit)
```

	Null Count	Null Percentage (%)
User_ID	0	0.000
Product_ID	0	0.000
Gender	0	0.000
Age	0	0.000
Occupation	0	0.000
City_Category	0	0.000
Stay_In_Current_City_Years	0	0.000
Marital_Status	0	0.000
Product_Category	0	0.000
Purchase	0	0.000

Next steps:

Generate code with null_audit

New interactive sheet

```
1 # Unique Records
2 walmart.nunique()
```

	0
User_ID	5891
Product_ID	3631
Gender	2
Age	7
Occupation	21
City_Category	3
Stay_In_Current_City_Years	5
Marital_Status	2
Product_Category	20
Purchase	18105

dtype: int64

```
1 # We have 5891 no. of Customers purchasing from available 3631 No. of different products at Walmart.
2 # The dataset provides data from 3 different City categories
3 # The available products have been categorized into 20 different Product Categories
```

```
1 # Convert categorical columns to 'category' data type for performance and semantic clarity
2 # From the info on dataset, we can say that other than "User_ID" & "Purchase" columns, all are categories
3
4 num_cols = ["User_ID","Purchase"]
5 cat_cols = walmart.columns[1:walmart.shape[1]-1]    # Purchase column is last column in dataset,
6 num_cols, cat_cols
```

```
(['User_ID', 'Purchase'],
 Index(['Product_ID', 'Gender', 'Age', 'Occupation', 'City_Category',
       'Stay_In_Current_City_Years', 'Marital_Status', 'Product_Category'],
      dtype='object'))
```

```
1 # Marital Status attribute conversion 0 = "Unmarried", 1 = "Married"
2 walmart['Marital_Status'] = walmart['Marital_Status'].replace({0:'Unmarried', 1:'Married'}).astype('category')
3 print(walmart['Marital_Status'].head(2))
4
5 # conversion of all object type columns to category
6 for col in cat_cols:
7     walmart[col] = walmart[col].astype('category')
8
9 # Create standardized Age Life Stage categorization
10 age_map = {
11     '0-17':'0-17',
12     '18-25':'18-25',
13     '26-35':'26-35',
14     '36-45':'36-50',
15     '46-50':'36-50',
16     '51-55':'51+',
17     '55+':'51+'
18 }
19 walmart['Age_life_stage'] = walmart['Age'] .map(age_map).astype('category')
20
21
```

```
0    Unmarried
1    Unmarried
Name: Marital_Status, dtype: category
Categories (2, object): ['Married', 'Unmarried']
```

```
1 for cols in cat_cols:
2     print(walmart[cols].value_counts())
3     print("====="*10)
```

```

5      12177
15      12165
11      11586
19      8461
13      7728
18      6622
9       6291
8       1546
Name: count, dtype: int64
=====
City_Category
B      231173
C      171175
A      147720
Name: count, dtype: int64
=====
Stay_In_Current_City_Years
1      193821
2      101838
3       95285
4+      84726
0       74398
Name: count, dtype: int64
=====
Marital_Status
Unmarried  324731
Married    225337
Name: count, dtype: int64
=====
Product_Category
5      150933
1      140378
8      113925
11     24287
2       23864
6       20466
3       20213
4       11753
16       9828
15       6290
13       5549
10       5125
12       3947
7        3721
18       3125
20       2550
19       1603
14       1523
17        578
9         410
Name: count, dtype: int64
=====

```

```

1 for cols in num_cols:
2     print(walmart[cols].value_counts())
3     print("====="*10)

```

```

User_ID
1001680    1026
1004277     979
1001941     898
1001181     862
1000889     823
...
1004991       7
1005391       7
1000094       7
1005608       7
1000708       6
Name: count, Length: 5891, dtype: int64
=====
Purchase
7011      191
7193      188
6855      187
6891      184
6960      183
...
18913      1
14361      1
5572       1
7290       1
15127      1
Name: count, Length: 18105, dtype: int64
=====

```

```

1 print("===== Updated Data Types =====\n")
2 print(walmart.dtypes)

```

```

===== Updated Data Types =====

User_ID          int64
Product_ID       category
Gender           category
Age             category

```

```
Occupation          category
City_Category       category
Stay_In_Current_City_Years  category
Marital_Status      category
Product_Category    category
Purchase            int64
Age_life_stage      category
dtype: object
```

```
1 walmart.info(memory_usage='deep')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   User_ID               550068 non-null  int64
1   Product_ID            550068 non-null  category
2   Gender                550068 non-null  category
3   Age                   550068 non-null  category
4   Occupation            550068 non-null  category
5   City_Category         550068 non-null  category
6   Stay_In_Current_City_Years  550068 non-null  category
7   Marital_Status        550068 non-null  category
8   Product_Category      550068 non-null  category
9   Purchase              550068 non-null  int64
10  Age_life_stage        550068 non-null  category
dtypes: category(9), int64(2)
memory usage: 14.0 MB
```

```
1 # Statistical Summary of Continuous and Categorical Features
2
3 print("==== Numerical Statistical Summary (Purchase) =====")
4 display(walmart['Purchase'].describe())
5
6 # Additional stats for Purchase
7 print('\n==== Additional Stats for Purchase =====\n')
8 mean_val = walmart['Purchase'].mean()
9 median_val = walmart['Purchase'].median()
10 std_val = walmart['Purchase'].std()
11 skew_val = walmart['Purchase'].skew()
12
13 print(f"Mean Purchase:      ${mean_val:.2f}")
14 print(f"Median Purchase:     ${median_val:.2f}")
15 print(f"Standard Deviation:    ${std_val:.2f}")
16 print(f"Mean - Median Gap:     ${mean_val - median_val:.2f} (Positive gap indicates right skew)")
17 print(f"Skewness:               {skew_val:.4f} (Moderately positive skewed)")
18
19 # Categorical Attributes Sunmmary
20 print("\n==== Categorical Attributes Summary =====")
21 display(walmart.describe(include=['category']))
```

===== Numerical Statistical Summary (Purchase) =====

Purchase	
count	550068.000
mean	9263.969
std	5023.065
min	12.000
25%	5823.000
50%	8047.000
75%	12054.000
max	23961.000

dtype: float64

===== Additional Stats for Purchase =====

Mean Purchase: \$9263.97
Median Purchase: \$8047.00
Standard Deviation: \$5023.07
Mean - Median Gap: \$1216.97 (Positive gap indicates right skew)
Skewness: 0.6001 (Moderately positive skewed)

===== Categorical Attributes Summary =====

	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Age_life_stage
count	550068	550068	550068	550068	550068	550068	550068	550068	550068
unique	3631	2	7	21	3	5	2	20	5
top	P00265242	M	26-35	4	B	1	Unmarried	5	26-35
freq	1880	414259	219587	72308	231173	193821	324731	150933	219587

```
1 walmart.groupby(['Gender'])['Purchase'].describe().T
```

Gender	F	M
count	135809.000	414259.000
mean	8734.566	9437.526
std	4767.233	5092.186
min	12.000	12.000
25%	5433.000	5863.000
50%	7914.000	8098.000
75%	11400.000	12454.000
max	23959.000	23961.000

2. Non-Graphical Analysis & Missing Value / Outlier Detection

We perform non-graphical analysis by evaluating unique counts, frequency tables, missing values, and detecting potential outliers via the Interquartile Range (IQR) technique.

```
1 # Non-Graphical Analysis: Unique counts and proportions
2 print("===== Unique Counts Per Feature =====")
3 walmart.nunique()
4
5 print("\n"+"="*100)
6 print("=== Frequency & Percentage Breakdown for Categorical Attributes ===")
7 for col in cat_cols:
8     freq_df = pd.DataFrame({
9         'Count': walmart[col].value_counts(),
10        'Percentage (%)': walmart[col].value_counts(normalize=True) * 100
11    })
12    print(f"\n--- Feature: {col} ---")
13    print(freq_df)
```

```

9      6291      1.144
8      1546      0.281

--- Feature: City_Category ---
      Count  Percentage (%)
City_Category
B      231173      42.026
C      171175      31.119
A      147720      26.855

--- Feature: Stay_In_Current_City_Years ---
      Count  Percentage (%)
Stay_In_Current_City_Years
1      193821      35.236
2      101838      18.514
3      95285      17.322
4+      84726      15.403
0      74398      13.525

--- Feature: Marital_Status ---
      Count  Percentage (%)
Marital_Status
Unmarried      324731      59.035
Married      225337      40.965

--- Feature: Product_Category ---
      Count  Percentage (%)
Product_Category
5      150933      27.439
1      140378      25.520
8      113925      20.711
11      24287      4.415
2      23864      4.338
6      20466      3.721
3      20213      3.675
4      11753      2.137
16      9828      1.787
15      6290      1.143
13      5549      1.009
10      5125      0.932
12      3947      0.718
7      3721      0.676
18      3125      0.568
20      2550      0.464
19      1603      0.291
14      1523      0.277
17      578      0.105
9      410      0.075

```

```

1 # Outlier Detection using IQR
2
3 Q1 = walmart['Purchase'].quantile(0.25)
4 Q3 = walmart['Purchase'].quantile(0.75)
5 IQR = Q3 - Q1
6 lower_bound = Q1 - IQR*1.5
7 upper_bound = Q3 + IQR*1.5
8
9 outliers = walmart[(walmart['Purchase']<lower_bound) | (walmart['Purchase'] > upper_bound)]
10 print('\n===== Outlier Analysis on Purchase Amount =====')
11 print(f'Q1 (25th percentile):    ${Q1:.2f}')
12 print(f'Q3 (75th percentile):    ${Q3:.2f}')
13 print(f'IQR:                    ${IQR:.2f}')
14 print(f'Lower Threshold:         ${lower_bound:.2f}')
15 print(f'Upper Threshold:         ${upper_bound:.2f}')
16 print(f'Outliers:                {len(outliers)}, records {len(outliers)/len(walmart)*100:.2f}% of total data')
17
18

```

```

===== Outlier Analysis on Purchase Amount =====
Q1 (25th percentile):    $5823.00
Q3 (75th percentile):    $12054.00
IQR:                    $6231.00
Lower Threshold:         $-3523.50
Upper Threshold:         $21400.50
Outliers:                2677, records 0.49% of total data

```

Key Observations

1. Zero Missing Values: The dataset is clean across all 550,068 rows with no missing entries.
2. Customer vs Product Scope: There are **5,891 unique customers** purchasing **3,631 distinct products** across 550,068 transactions.
3. Demographics:
 - Gender: Transactions are 75.3% Male (414,259 transactions) and 24.7% Female (135,809 transactions).
 - Age: The customers in life stage 26–35 age group dominate sales volume (39.9%), followed by 36–45 (20.0%) and 18–25 (18.1%).

- City Category: City B accounts for 42.0% of transactions, City C for 31.1%, and City A for 26.8%.
- Marital Status: 59.0% of customers are by unmarried individuals and 41.0% by married individuals.

4. Outlier Interpretation:

- The IQR rule identifies 2,677 transactions (> \$21,400.50) as statistical outliers (only 0.49% of data).
- Business Decision: These purchases represent genuine high-ticket transactions (such as major electronics, televisions, appliances) typical during Black Friday doorbuster sales. *They must NOT be removed*, as retaining them is essential for accurate revenue modeling and inventory valuation.

1 Start coding or [generate](#) with AI.

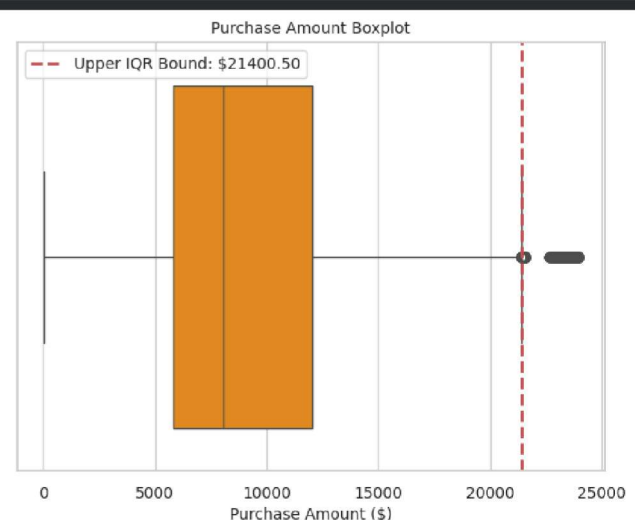
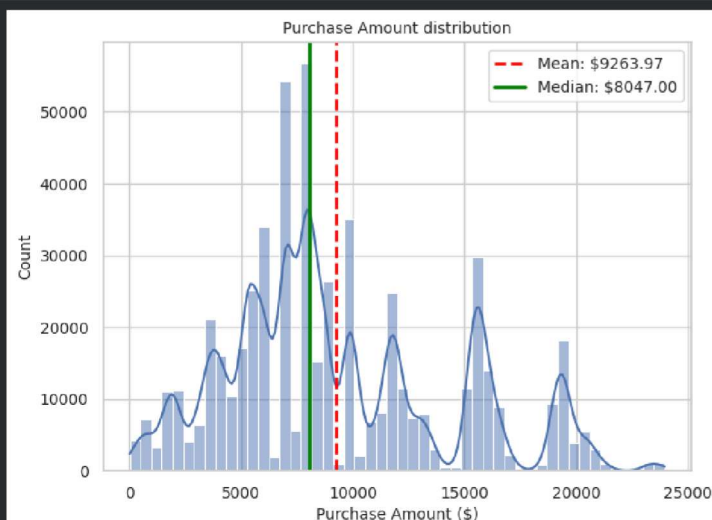
3. Visual Analysis (Univariate, Bivariate)

In this section, we visually explore distributions, demographic patterns, relationships between features, and correlations.

```
1 walmart.head()
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase	Age
0	1000001	P00069042	F	0-17	10	A	2	Unmarried	3	8370	
1	1000001	P00248942	F	0-17	10	A	2	Unmarried	1	15200	
2	1000001	P00087842	F	0-17	10	A	2	Unmarried	12	1422	

```
1 # 3.1 Univariate Analysis: Continuous Variable (Purchase)
2
3 plt.subplots(1,2,figsize=(15,5))
4 plt.subplot(1,2,1)
5 # Distribution plot
6 ax = sns.histplot(walmart['Purchase'], kde = True, bins = 50, color = 'b')
7 plt.axvline(mean_val, color='red', linestyle='--', linewidth=2, label=f'Mean: ${mean_val:.2f}')
8 plt.axvline(median_val, color='green', linestyle='--', linewidth=2, label=f'Median: ${median_val:.2f}')
9 plt.title("Purchase Amount distribution")
10 plt.xlabel('Purchase Amount ($)')
11 plt.legend()
12
13 plt.subplot(1,2,2)
14 ax = sns.boxplot(x = walmart['Purchase'], color = 'darkorange')
15 plt.axvline(upper_bound, color = 'r', linestyle='--', linewidth=2, label = f'Upper IQR Bound: ${upper_bound:.2f}')
16 plt.title('Purchase Amount Boxplot')
17 plt.xlabel('Purchase Amount ($)')
18 plt.legend()
19
20 plt.show()
```



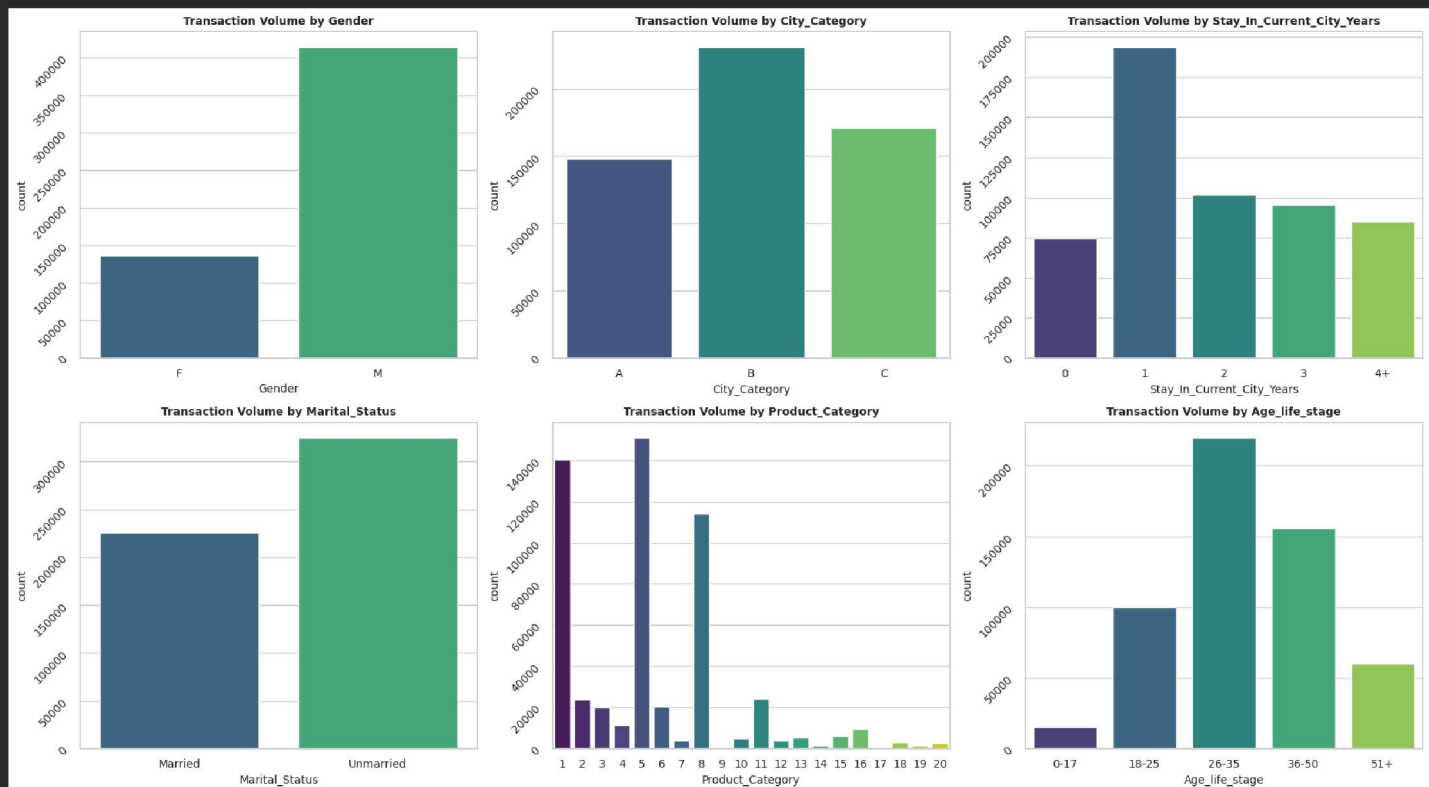
```
1 print(len(cat_cols))
2 walmart.head(1)
```

8

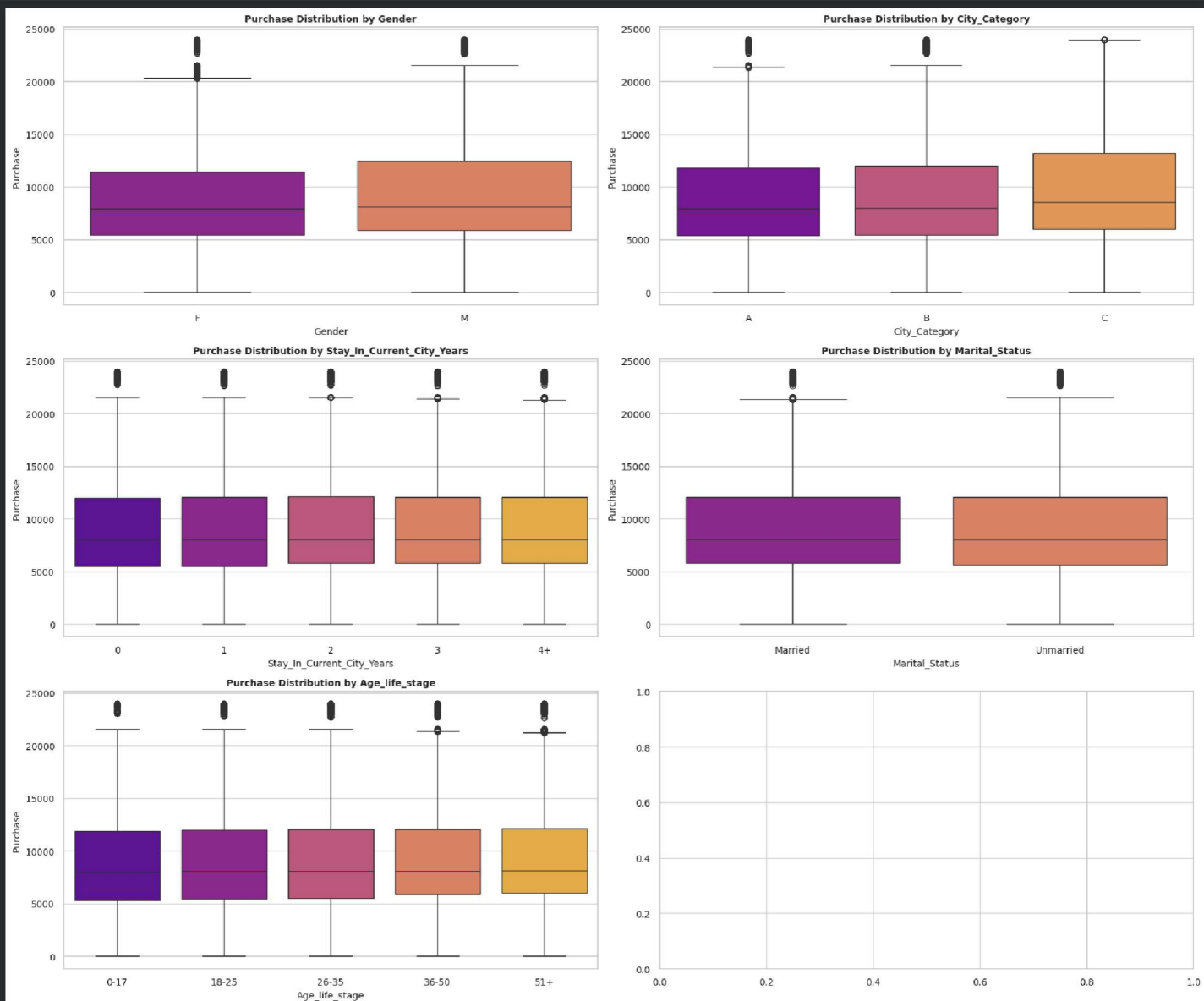
	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase	Age
--	---------	------------	--------	-----	------------	---------------	----------------------------	----------------	------------------	----------	-----

0	1000001	P00069042	F	0-17		A	2	Unmarried	3	8370	
---	---------	-----------	---	------	--	---	---	-----------	---	------	--

```
1 # # 3.2 Univariate Analysis: Categorical Demographic Variables
2
3 plt.subplots(2, 3, figsize=(18, 10))
4 cat_plots = ['Gender', 'City_Category', 'Stay_In_Current_City_Years', 'Marital_Status', 'Product_Category', 'Age_life_st
5 i = 1
6 for col in cat_plots:
7     plt.subplot(2,3,i)
8     sns.countplot(data = walmart, x = col, palette = 'viridis',hue = col, legend = False)
9     plt.title(f'Transaction Volume by {col}', fontweight = 'bold')
10    plt.yticks(rotation = 45)
11    i+=1
12
13 plt.tight_layout()
14 plt.show()
```



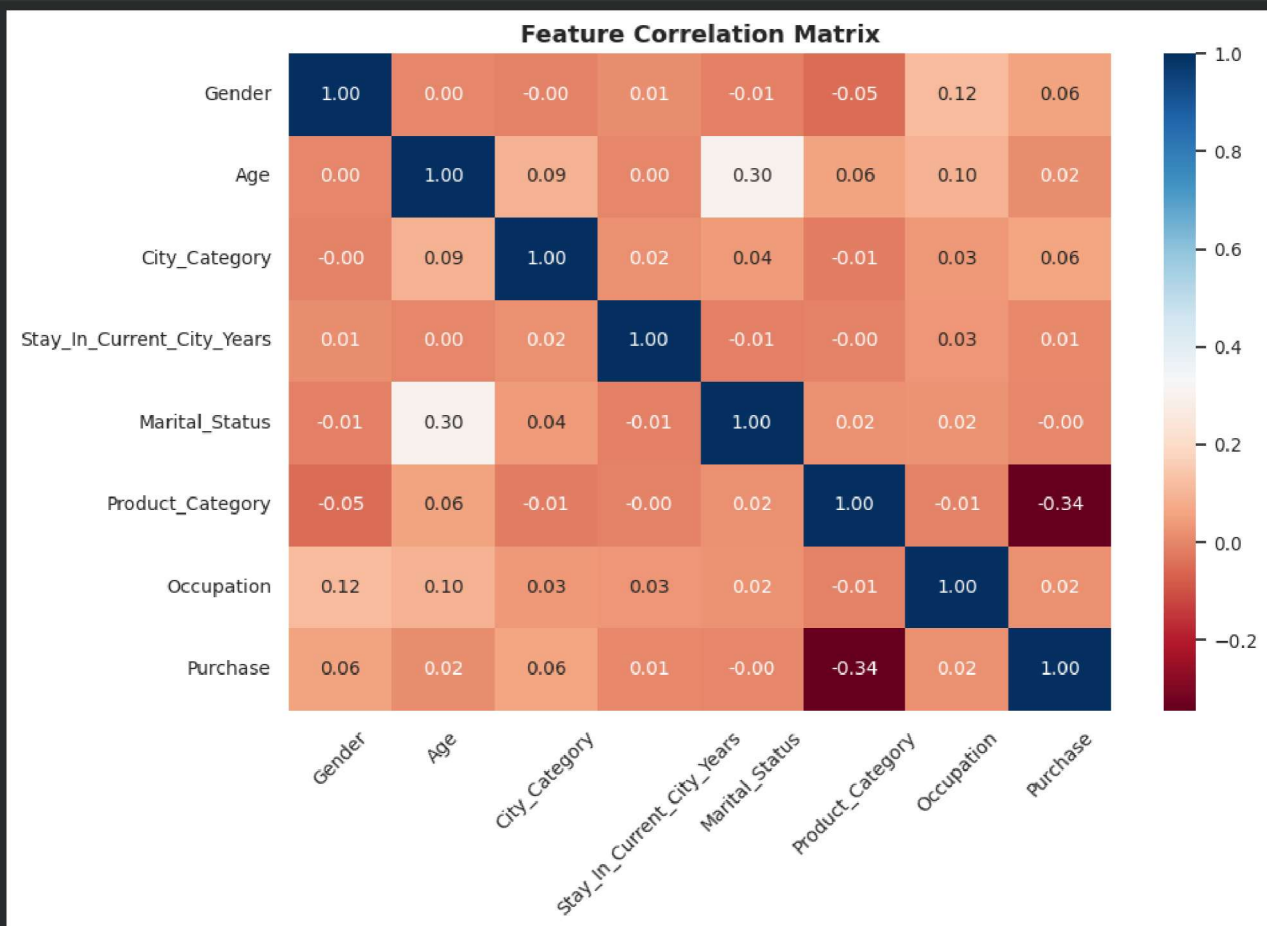
```
1 # 3.3 Bivariate Analysis: Purchase vs Categorical Predictors
2
3 plt.subplots(3, 2, figsize=(18, 15))
4 cat_boxes = ['Gender', 'City_Category', 'Stay_In_Current_City_Years', 'Marital_Status', 'Age_life_stage']
5 i = 1
6 for col in cat_boxes:
7     plt.subplot(3,2,i)
8     sns.boxplot(data = walmart, x = col, y = 'Purchase', palette = 'plasma',hue = col, legend = False)
9     plt.title(f'Purchase Distribution by {col}', fontweight = 'bold')
10    i+=1
11
12 plt.tight_layout()
13 plt.show()
```



```

1 # 3.4 Correlation Analysis
2 corr_df = pd.DataFrame()
3 corr_df['Gender'] = (walmart['Gender'].astype(str) == 'M').astype(int) # Map Gender Male = 1, Female = 0
4 age_codes = {'0-17': 0, '18-25': 1, '26-35': 2, '36-50': 3, '51+': 4}
5 corr_df['Age'] = walmart['Age_life_stage'].astype(str).map(age_codes).astype(int)
6 city_codes = {'A':1, 'B':2, 'C':3}
7 corr_df['City_Category'] = walmart['City_Category'].astype(str).map(city_codes).astype(int)
8 corr_df['Stay_In_Current_City_Years'] = walmart['Stay_In_Current_City_Years'].astype(str).replace('4+', '4').astype(int)
9 corr_df['Marital_Status'] = walmart['Marital_Status'].astype(str).replace({'Unmarried': '0', 'Married': '1'}).astype(int)
10 corr_df['Product_Category'] = walmart['Product_Category'].astype(int)
11 corr_df['Occupation'] = walmart['Occupation'].astype(int)
12 corr_df['Purchase'] = walmart['Purchase'].astype(int)
13
14 plt.figure(figsize=(10, 7))
15 sns.heatmap(corr_df.corr(), annot=True, fmt=".2f", cmap = 'RdBu') #vmin=-0.15, vmax=0.15)
16 plt.title('Feature Correlation Matrix', fontsize=13, fontweight='bold')
17
18 plt.xticks(rotation = 45)
19 plt.tight_layout()
20 plt.show()

```



Insights from Visual Analysis

- Purchase Distribution:** The purchase histogram displays peaks around amounts 5000, 8000, 12000, 15000, and 19000-20000. These correspond to distinct standardized retail price tiers (e.g. entry accessories, mid-range appliances, premium electronics).
- Gender Spend Difference:** Male Customer transactions exhibit a higher upper quartile and higher median spend compared to female customer transactions.
- Marital Status Invariance:** The purchase distributions for married vs unmarried customers are identical across quartiles, medians, and spreads.
- Age Groups:** Average spend increases steadily across the life stages, with customers aged 51+ exhibiting the highest median spend.
- Low Linear Correlation:** Correlation between attributes and individual transaction amounts are ($|r| < 0.1$), indicating that transaction spending is governed by non-linear categorical preferences and basket selection rather than simple monotonic linear relationships. The closest to near linear correlatin is between Product Category and Purchase ($r = -0.34$), although a *-ve* correlation.

4. Answering Core Questions

Question 1: Are women spending more money per transaction than men? Why or why not?

To address this question with mathematical rigor, we analyze the metrics at two fundamental levels:

- Transaction Level:** Amount spent per individual transaction.
- Customer Level:** Total spending, transaction count, and average spend per individual user.
- Product Category Mix**

1 Start coding or [generate](#) with AI.

```
1 # 4.1 Transaction-Level Metrics by Gender
2 gender_tx_summary = walmart.groupby('Gender', observed=False)['Purchase'].agg(
3     Transaction_Count='count',
4     Mean_Spend='mean',
5     Std_Dev='std',
6     Median_Spend='median'
7 ).reset_index()
8
```

```

9 gender_tx_summary['Share_of_Transactions_%'] = (gender_tx_summary['Transaction_Count'] / len(walmart)) * 100
10 print("==== Transaction-Level Summary by Gender =====")
11 display(gender_tx_summary)
12 print('\n')
13
14 # 4.2 Customer-Level Metrics by Gender
15 user_agg = walmart.groupby(['User_ID', 'Gender'], observed=False).agg(
16     Total_Spend=('Purchase', 'sum'),
17     Transaction_Count=('Purchase', 'count'),
18     Avg_Spend_Per_Tx=('Purchase', 'mean')
19 ).reset_index()
20 # print(user_agg, end='\n')
21 # print('\n')
22
23 user_summary = user_agg.groupby('Gender', observed=False).agg(
24     Customer_Count=('User_ID', 'count'),
25     Mean_Total_Spend=('Total_Spend', 'mean'),
26     Median_Total_Spend=('Total_Spend', 'median'),
27     Mean_Tx_Count=('Transaction_Count', 'mean'),
28     Mean_Avg_Spend_Per_Tx=('Avg_Spend_Per_Tx', 'mean')
29 ).reset_index()
30
31 user_summary['Customer_Share_%'] = (user_summary['Customer_Count'] / user_agg['User_ID'].nunique()) * 100
32 print("\n==== Customer-Level Summary by Gender =====")
33 display(user_summary)
34 # print('\n')
35
36 # 4.3 Product Category Breakdown
37 cat_gender = walmart.groupby(['Product_Category', 'Gender'], observed=False)['Purchase'].agg(
38     Count='count',
39     Mean='mean'
40 ).unstack()
41 # print(cat_gender, end = '\n')
42 # print('\n')
43
44 top_10_cats = walmart['Product_Category'].value_counts().head(10).index.sort_values()
45 # print(top_10_cats)
46 # print('\n')
47 cat_sub = cat_gender.loc[top_10_cats]
48 # print(cat_sub, end = '\n')
49 # print('\n')
50
51 print("\n==== Mean Spend & Share by Product Category (Top 10 Categories) =====")
52 cat_df = pd.DataFrame({
53     'Female Mean ($)': cat_sub['Mean']['F'],
54     'Male Mean ($)': cat_sub['Mean']['M'],
55     'Difference ($)': cat_sub['Mean']['M'] - cat_sub['Mean']['F'],
56     'Female Tx Count': cat_sub['Count']['F'],
57     'Male Tx Count': cat_sub['Count']['M'],
58     'Female Cat Share (%)': (cat_sub['Count']['F'] / len(walmart[walmart['Gender']=='F'])) * 100,
59     'Male Cat Share (%)': (cat_sub['Count']['M'] / len(walmart[walmart['Gender']=='M'])) * 100
60 })

```

==== Transaction-Level Summary by Gender =====

	Gender	Transaction_Count	Mean_Spend	Std_Dev	Median_Spend	Share_of_Transactions_%
0	F	135809	8734.566	4767.233	7914.000	24.689
1	M	414259	9437.526	5092.186	8098.000	75.311

==== Customer-Level Summary by Gender =====

	Gender	Customer_Count	Mean_Total_Spend	Median_Total_Spend	Mean_Tx_Count	Mean_Avg_Spend_Per_Tx	Customer_Share_%
0	F	5891	201363.545	0.000	23.054	8965.198	100.000
1	M	5891	663653.047	308629.000	70.321	9806.868	100.000

==== Mean Spend & Share by Product Category (Top 10 Categories) =====

Product_Category	Female Mean (\$)	Male Mean (\$)	Difference (\$)	Female Tx Count	Male Tx Count	Female Cat Share (%)	Male Cat Share (%)
1	13597.163	13608.165	11.002	24831	115547	18.284	27.892
2	11407.497	11203.591	-203.906	5658	18206	4.166	4.395
3	10262.657	10026.550	-236.107	6006	14207	4.422	3.429
4	2454.852	2273.513	-181.339	3639	8114	2.679	1.959
5	6307.240	6214.231	-93.009	41961	108972	30.897	26.305
6	15596.428	15907.851	311.423	4559	15907	3.357	3.840
8	7499.925	7498.554	-1.370	33558	80367	24.710	19.400
11	4676.372	4687.425	11.053	4739	19548	3.489	4.719
15	14695.327	14797.431	102.104	1046	5244	0.770	1.266
16	14681.491	14793.384	111.893	2402	7426	1.769	1.793

Next steps:

Generate code with gender_tx_summary

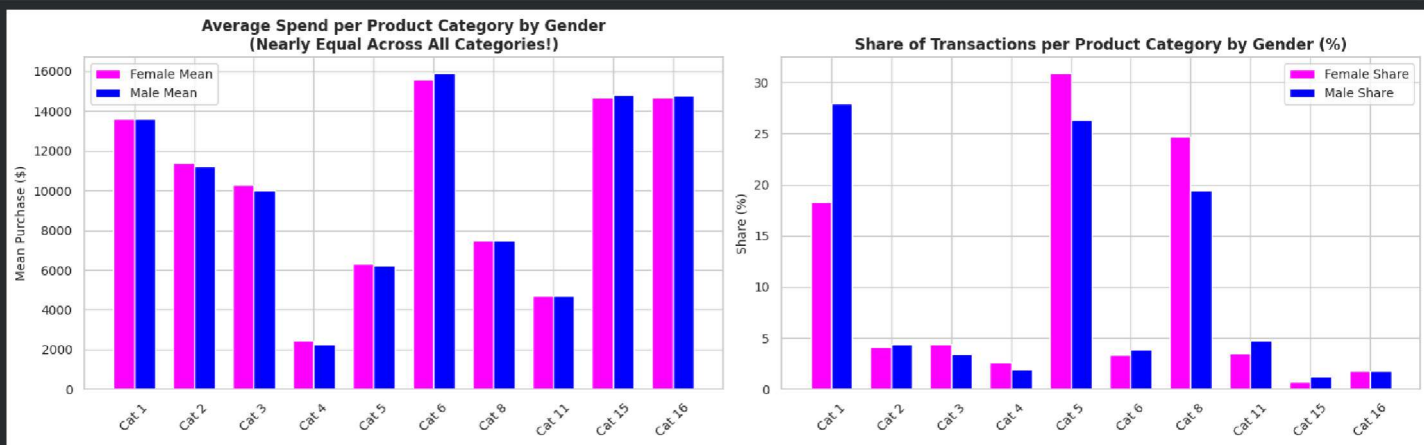
New interactive sheet

Generate code with user_summary

New interactive sheet

Generate code with c

```
1 # # Visualizing the Product Mix
2
3 plt.subplots(1,2, figsize =(16, 5) )
4 x_idx = np.arange(len(top_10_cats))
5 width = 0.35
6
7 # Chart A: Mean spend per category
8 plt.subplot(1,2,1)
9 plt.bar(x_idx - width/2, cat_df['Female Mean ($)'], width, label='Female Mean', color='magenta')
10 plt.bar(x_idx + width/2, cat_df['Male Mean ($)'], width, label='Male Mean', color='blue')
11
12 plt.title('Average Spend per Product Category by Gender\n(Nearly Equal Across All Categories!)', fontsize=12, fontwe
13 plt.ylabel('Mean Purchase ($)')
14 plt.xticks(x_idx, [f"Cat {c}" for c in top_10_cats], rotation=45)
15 plt.legend()
16
17 # Chart B: Share of Transactions per category by Gender
18 plt.subplot(1,2,2)
19 plt.bar(x_idx - width/2, cat_df['Female Cat Share (%)'], width, label='Female Share', color='magenta')
20 plt.bar(x_idx + width/2, cat_df['Male Cat Share (%)'], width, label='Male Share', color='blue')
21
22 plt.title('Share of Transactions per Product Category by Gender (%)', fontsize=12, fontweight='bold')
23 plt.ylabel('Share (%)')
24 plt.xticks(x_idx, [f"Cat {c}" for c in top_10_cats], rotation=45)
25 plt.legend()
26
27 plt.tight_layout()
28 plt.show()
```



Answer to Question 1: Do Women Spend More Than Men?

Conclusion: NO. Women do NOT spend more money per transaction than men.

- Per Transaction: Men spend an average of \$9,437.53, whereas women spend an average of \$8,734.57. Male spend is higher by \$702.96 per transaction (+8.05%).
- Per Customer: Male customers generate an average total spend of \$925,344 across 98.0 transactions, whereas female customers generate \$712,024 across 81.5 transactions.

Why do men spend more?

- Equal Purchasing Power Within Categories: When we inspect individual product categories, men and women spend virtually identical amounts:
 - In Product Category 1: Women spend \$13,597.16 vs Men \$13,608.16 (difference of only \$11.00).
 - In Product Category 5: Women spend \$6,307.24 vs Men \$6,214.23 (women actually spend \$93 more).
 - In Product Category 8: Women spend \$7,499.92 vs Men \$7,498.55 (difference of \$1.37).
- The Product Mix Shift:
 - Men allocate 27.89% of all their purchases to Product Category 1 (items averaging \$13,600+), compared to only 18.28% for women.
 - Women allocate a higher share of their transactions to lower-ticket categories like Category 5 (30.90% for women vs 26.31% for men).
 - Therefore, the higher overall spending of men is driven entirely by product selection mix, not because men pay more for the same goods.

1 Start coding or [generate](#) with AI.

Question 2: How does sample size affect the distribution of mean expenses for male and female customers?

Question 3: Are confidence intervals of average male and female spending overlapping? How can Walmart leverage this conclusion?

5. Confidence Interval & Central Limit Theorem

The dataset represents a sample of Walmart transactions. We use this sample to estimate the average purchase amount for the larger male and female customer populations.

We first calculate Confidence Intervals for male and female customers and then use different sample sizes to understand how the distribution of sample means changes.

```
1 # 5.1 95% Confidence Interval for Purchase Amount by Gender
2
3 ci_results = []
4
5 for gender in ['F', 'M']:
6     sample = walmart[walmart['Gender'] == gender]['Purchase']
```

```

7     sample_mean = sample.mean()
8     sample_std = sample.std()
9     sample_size = len(sample)
10    standard_error = sample_std / np.sqrt(sample_size)
11    ci = stats.norm.interval(confidence=0.95, loc=sample_mean, scale=standard_error)
12
13    ci_results.append([gender, sample_size, sample_mean, ci[0], ci[1]])
14
15 ci_df = pd.DataFrame(ci_results, columns=[
16     'Gender',
17     'Sample Size',
18     'Mean Purchase',
19     '95% CI Lower',
20     '95% CI Upper'
21 ])
22 )
23
24 display(ci_df)

```

	Gender	Sample Size	Mean Purchase	95% CI Lower	95% CI Upper
0	F	135809	8734.566	8709.212	8759.920
1	M	414259	9437.526	9422.019	9453.033

Next steps: [Generate code with ci_df](#) [New interactive sheet](#)

5.2 Central Limit Theorem — Effect of Sample Size

To understand how sample size affects the sample mean, we take repeated random samples of different sizes and calculate the mean purchase amount for each sample.

Sample sizes used:

- 30
- 100
- 500
- 1000

We repeat the sampling process 1,000 times for each sample size.

```

1 # 5.3 CLT - Female Customers
2
3 sample_sizes = [30, 100, 500, 1000]
4 number_of_samples = 1000
5
6 female_purchase = walmart[walmart['Gender'] == 'F']['Purchase']
7 female_sample_means = {}
8
9 for n in sample_sizes:
10     sample_means = []
11
12     for i in range(number_of_samples):
13         sample = female_purchase.sample(n=n, replace=True)
14         sample_means.append(sample.mean())
15     female_sample_means[n] = sample_means

```

```

1 # 5.4 CLT - Male Customers
2
3 male_purchase = walmart[walmart['Gender'] == 'M']['Purchase']
4 male_sample_means = {}
5 for n in sample_sizes:
6     sample_means = []
7
8     for i in range(number_of_samples):
9         sample = male_purchase.sample(n=n, replace=True)
10        sample_means.append(sample.mean())
11    male_sample_means[n] = sample_means

```

```

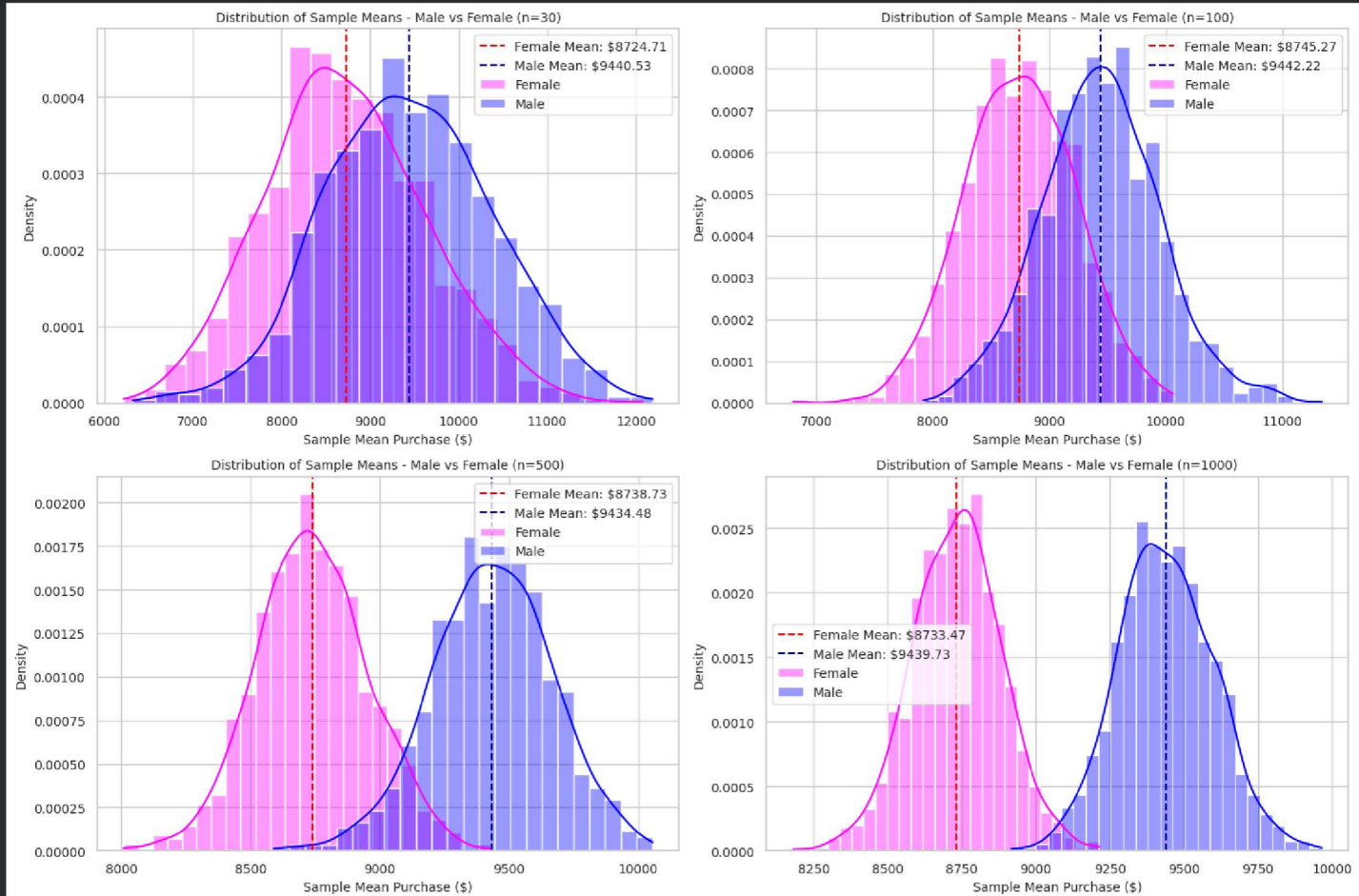
1 # CLT - Male vs Female for Different Sample Sizes
2 i = 1
3 plt.subplots(2,2,figsize=(15,10))
4 for n in sample_sizes:
5
6     plt.subplot(2,2,i)
7     sns.histplot(female_sample_means[n], kde=True, color='magenta', label='Female', stat='density', alpha=0.4)
8     sns.histplot(male_sample_means[n], kde=True, color='blue', label='Male', stat='density', alpha=0.4)

```

```

9     plt.axvline(np.mean(female_sample_means[n]), color = 'red', linestyle='--', linewidth=1.5,
10               label=f'Female Mean: ${np.mean(female_sample_means[n]):.2f}')
11     plt.axvline(np.mean(male_sample_means[n]), color = 'darkblue', linestyle='--', linewidth=1.5,
12               label=f'Male Mean: ${np.mean(male_sample_means[n]):.2f}')
13
14     plt.title(f'Distribution of Sample Means - Male vs Female (n={n})')
15     plt.xlabel('Sample Mean Purchase ($)')
16     plt.ylabel('Density')
17     plt.legend(loc='best')
18     i+=1
19
20 plt.tight_layout()
21 plt.show()

```



```

1 # 5.5 CLT Summary - Male vs Female
2
3 clt_summary = []
4
5 for n in sample_sizes:
6
7     clt_summary.append([
8         n,
9         np.mean(female_sample_means[n]),
10        np.std(female_sample_means[n]),
11        np.mean(male_sample_means[n]),
12        np.std(male_sample_means[n])
13    ])
14
15 clt_summary_df = pd.DataFrame(
16     clt_summary,
17     columns=[
18         'Sample Size',

```

```

19     'Female Mean',
20     'Female Std Dev',
21     'Male Mean',
22     'Male Std Dev'
23 ]
24 )
25
26 display(clt_summary_df)

```

	Sample Size	Female Mean	Female Std Dev	Male Mean	Male Std Dev
0	30	8724.713	902.019	9440.527	930.822
1	100	8745.269	475.290	9442.218	504.825
2	500	8738.735	214.967	9434.478	232.305
3	1000	8733.467	150.078	9439.728	160.112



Next steps:

[Generate code with clt_summary_df](#)

[New interactive sheet](#)

```

1 # 5.6 Comparing 90%, 95% and 99% Confidence Intervals
2
3 confidence_levels = [0.90, 0.95, 0.99]
4
5 ci_comparison = []
6
7 for gender in ['F', 'M']:
8
9     sample = walmart[walmart['Gender'] == gender]['Purchase']
10    sample_mean = sample.mean()
11    standard_error = sample.std() / np.sqrt(len(sample))
12
13    for confidence_level in confidence_levels:
14        ci = stats.norm.interval(confidence=confidence_level, loc=sample_mean, scale=standard_error)
15
16        ci_comparison.append([gender, confidence_level * 100, ci[0], ci[1]])
17
18 ci_comparison_df = pd.DataFrame(
19     ci_comparison,
20     columns=[
21         'Gender',
22         'Confidence Level (%)',
23         'CI Lower',
24         'CI Upper'
25     ]
26 )
27
28 display(ci_comparison_df)

```

	Gender	Confidence Level (%)	CI Lower	CI Upper
0	F	90.000	8713.288	8755.844
1	F	95.000	8709.212	8759.920
2	F	99.000	8701.245	8767.887
3	M	90.000	9424.512	9450.540
4	M	95.000	9422.019	9453.033
5	M	99.000	9417.147	9457.905



Next steps:

[Generate code with ci_comparison_df](#)

[New interactive sheet](#)

1 Start coding or [generate](#) with AI.

Observations

1. The 90% confidence interval is the narrowest.
2. The 95% confidence interval is wider than the 90% interval.
3. The 99% confidence interval is the widest.
4. As the confidence level increases, the confidence interval becomes wider.
5. This means that a higher level of confidence requires a wider range around the sample average.

5.7 95% Confidence Interval - Male vs Female

We calculate the 95% confidence interval for the average purchase amount of Male and Female customers and compare the intervals to understand whether their average spending is clearly different.

```
1 # 95% Confidence Interval - Male vs Female
2
3 gender_ci = {}
4
5 for gender in ['F', 'M']:
6
7     purchase = walmart[walmart['Gender'] == gender]['Purchase']
8
9     mean = purchase.mean()
10    std_error = purchase.std() / np.sqrt(len(purchase))
11
12    gender_ci[gender] = stats.norm.interval(
13        confidence=0.95,
14        loc=mean,
15        scale=std_error
16    )
17
18 gender_ci_df = pd.DataFrame(
19     gender_ci,
20     index=['Lower', 'Upper']
21 ).T
22
23 gender_ci_df['Mean'] = [
24     walmart[walmart['Gender'] == 'F']['Purchase'].mean(),
25     walmart[walmart['Gender'] == 'M']['Purchase'].mean()
26 ]
27
28 display(gender_ci_df)
```

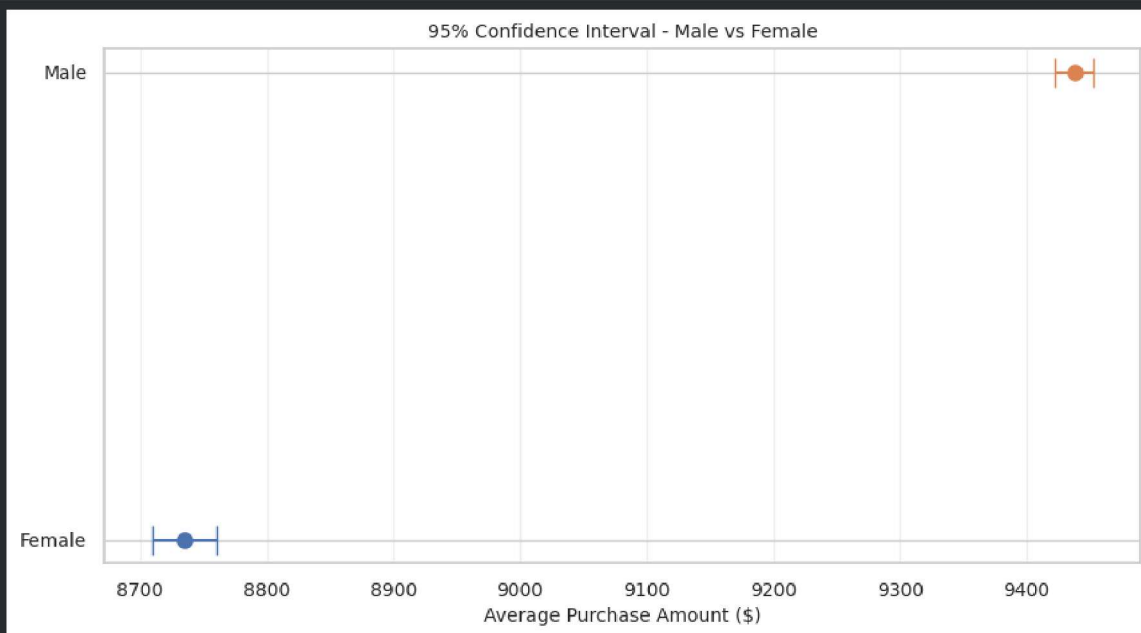
	Lower	Upper	Mean
F	8709.212	8759.920	8734.566
M	9422.019	9453.033	9437.526

Next steps:

[Generate code with gender_ci_df](#)

[New interactive sheet](#)

```
1 # 95% Confidence Interval - Male vs Female
2
3 plt.figure(figsize=(10, 5))
4
5 for i, gender in enumerate(['F', 'M']):
6
7     mean = gender_ci_df.loc[gender, 'Mean']
8     lower = gender_ci_df.loc[gender, 'Lower']
9     upper = gender_ci_df.loc[gender, 'Upper']
10
11     plt.errorbar(
12         mean,
13         i,
14         xerr=[[mean - lower], [upper - mean]],
15         fmt='o',
16         capsize=8,
17         markersize=8,
18         label=gender
19     )
20
21 plt.yticks([0, 1], ['Female', 'Male'])
22 plt.xlabel('Average Purchase Amount ($)')
23 plt.title('95% Confidence Interval - Male vs Female')
24 plt.grid(axis='x', alpha=0.3)
25
26 plt.show()
```



Business Conclusion on Confidence Intervals & Overlap:

1. Zero Overlap at All Confidence Levels:

- Even at the most conservative 99% Confidence Level, the Female interval is [\$8,701.24, \$8,767.89] while the Male interval is [\$9,417.15, \$9,457.90].
- The gap between the upper bound of female spend and the lower bound of male spend is \$649.26.

2. How Walmart Can Leverage This Conclusion:

- Targeted Black Friday Campaigns: Since men spend more per transaction and represent 75.3% of transaction volume, Walmart should maintain aggressive marketing and doorbuster positioning for high-ticket male-leaning categories (electronics, gaming, tools).
- Upselling & Basket Building for Women: Since women have equal spending within categories but lower overall transaction size due to lower-ticket category preference, Walmart should deploy cross-category bundling (e.g. bundling high-margin electronics with lifestyle/apparel items) to elevate female basket sizes.
- Inventory Allocation: Allocate prime floor space and warehouse safety stock in higher proportion to high-ticket Category 1 goods to meet heavy male demand without stockouts.

Question 4: Does purchase behavior differ significantly between Married and Unmarried customers?

6. Demographic Deep-Dive: Married vs Unmarried

We apply the exact same CLT and Confidence Interval methodology to compare **Unmarried** (`Marital_Status = 0`) and **Married** (`Marital_Status = 1`) customers across 90%, 95%, and 99% confidence levels.

```
1 walmart['Marital_Status'].head(2)
```

Marital_Status

```
0 Unmarried
1 Unmarried
```

dtype: category

```
1 # Purchase Summary - Married vs Unmarried
2
3 marital_summary = walmart.groupby('Marital_Status')['Purchase'].agg(
4     ['count', 'mean', 'std']
5 ).reset_index()
6
7 display(marital_summary)
8
9 # Confidence Intervals - Married vs Unmarried
10
11 marital_ci = {}
```

```

12
13 for status in ['Unmarried', 'Married']:
14     purchase = walmart[walmart['Marital_Status'] == status]['Purchase']
15
16     mean = purchase.mean()
17     std_error = purchase.std() / np.sqrt(len(purchase))
18
19     marital_ci[status] = {}
20
21     for confidence in [0.90, 0.95, 0.99]:
22         marital_ci[status][confidence] = stats.norm.interval(confidence=confidence,
23                                                                loc=mean, scale=std_error)

```

	Marital_Status	count	mean	std
0	Married	225337	9261.175	5016.897
1	Unmarried	324731	9265.908	5027.348

Next steps:

[Generate code with marital_summary](#)

[New interactive sheet](#)

```

1 # Display Confidence Intervals
2
3 marital_ci_summary = []
4
5 for confidence in [0.90, 0.95, 0.99]:
6
7     marital_ci_summary.append([
8         f'{int(confidence * 100)}%',
9         marital_ci['Unmarried'][confidence][0],
10        marital_ci['Unmarried'][confidence][1],
11        marital_ci['Married'][confidence][0],
12        marital_ci['Married'][confidence][1]
13    ])
14
15 marital_ci_df = pd.DataFrame(
16     marital_ci_summary,
17     columns=[
18         'Confidence Level',
19         'Unmarried Lower',
20         'Unmarried Upper',
21         'Married Lower',
22         'Married Upper'
23     ]
24 )
25
26 display(marital_ci_df)

```

	Confidence Level	Unmarried Lower	Unmarried Upper	Married Lower	Married Upper
0	90%	9251.396	9280.419	9243.791	9278.558
1	95%	9248.616	9283.199	9240.460	9281.889
2	99%	9243.183	9288.632	9233.952	9288.398

Next steps:

[Generate code with marital_ci_df](#)

[New interactive sheet](#)

```

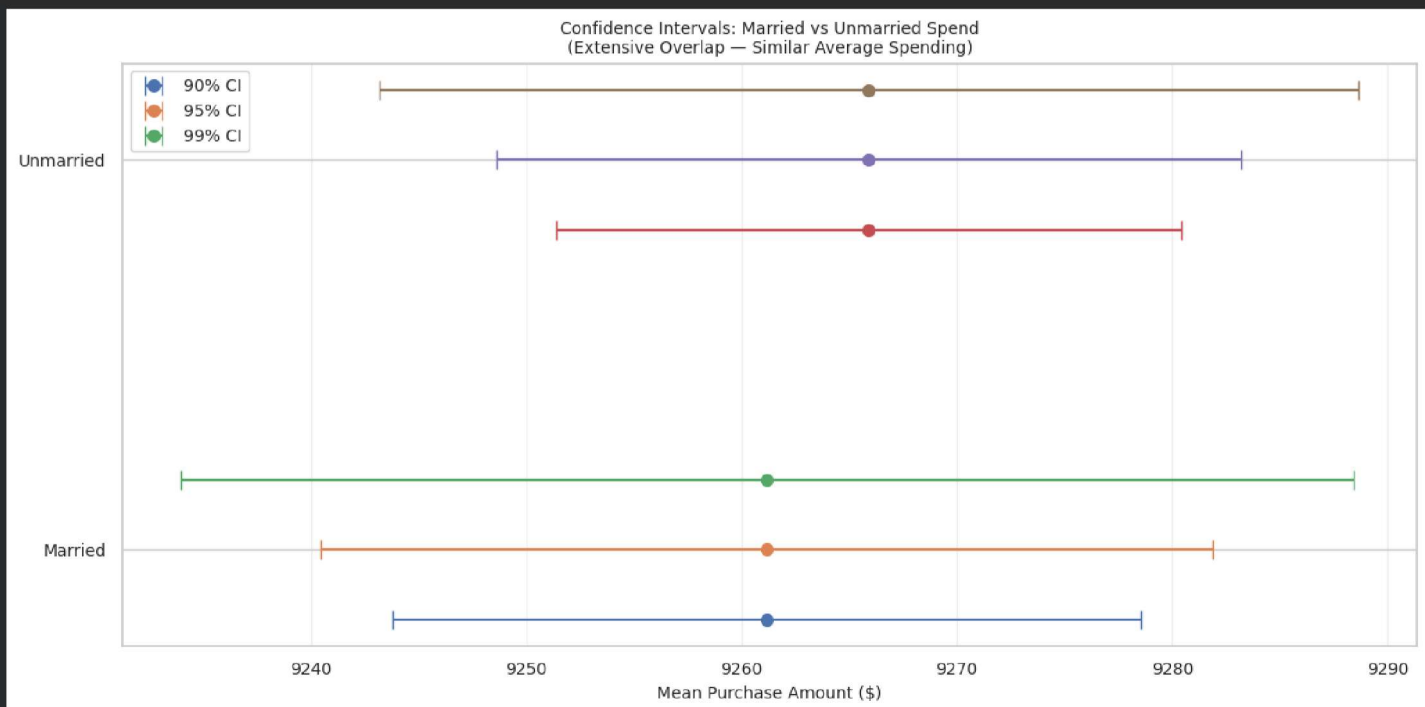
1 # Confidence Intervals - Married vs Unmarried
2
3 plt.figure(figsize=(12, 6))
4 confidence_levels = [0.90, 0.95, 0.99]
5
6 for i, status in enumerate(['Married', 'Unmarried']):
7     mean = walmart[walmart['Marital_Status'] == status]['Purchase'].mean()
8     for j, confidence in enumerate(confidence_levels):
9         lower = marital_ci[status][confidence][0]
10        upper = marital_ci[status][confidence][1]
11
12        y_position = i + (j - 1) * 0.18
13
14        plt.errorbar(mean, y_position, xerr=[[mean - lower], [upper - mean]],
15                    fmt='o', capsize=6, markersize=7,
16                    label=f'{int(confidence * 100)}% CI' if i == 0 else None)
17
18 plt.yticks([0, 1], ['Married', 'Unmarried'])
19 plt.xlabel('Mean Purchase Amount ($)')
20 plt.ylabel('')
21 plt.title('Confidence Intervals: Married vs Unmarried Spend\n')

```

```

22 '(Extensive Overlap – Similar Average Spending)'
23 )
24
25 plt.legend()
26 plt.grid(axis='x', alpha=0.3)
27
28 plt.tight_layout()
29 plt.show()

```



Findings for Married vs Unmarried Customers

1. Negligible Spend Difference: Unmarried customers spend an average of \$9,265.91 while married customers spend \$9,261.17. The difference is a mere \$4.74 per transaction.
2. Confidence Intervals Overlap Completely:
 - At 95% Confidence: Unmarried [\$9,248.62, \$9,283.20] vs Married [\$9,240.46, \$9,281.88].
 - At 99% Confidence: Unmarried [\$9,243.18, \$9,288.63] vs Married [\$9,233.95, \$9,288.40].
3. Actionable Business Takeaway: Walmart should NOT segment pricing, promotional discounting, or general catalog curation based on marital status. Budget allocation should not be wasted on tailoring separate Black Friday pricing tiers for married vs single shoppers.

Question 5: How does purchase amount vary across customer life stages?

7. Demographic Deep-Dive: Age Life Stages

We categorize age into the 5 business life stages:

- **0-17** : Minors / Teens (Dependent)
- **18-25** : Young Adults / College / Entry-level
- **26-35** : Prime Working Age / Early Career
- **36-50** : Mature Professionals / Families
- **51+** : Empty Nesters & Seniors (Peak Disposable Income)

We compute descriptive metrics, sampling distributions, 90%/95%/99% Confidence Intervals.

```

1 # Purchase Summary - Age Life Stages
2
3 age_summary = walmart.groupby('Age_life_stage', observed=True)['Purchase'].agg(
4     ['count', 'mean', 'std']
5 ).reset_index()
6
7 display(age_summary)

```

	Age_life_stage	count	mean	std
0	0-17	15102	8933.465	5111.114
1	18-25	99660	9169.664	5034.322
2	26-35	219587	9252.691	5010.527
3	36-50	155714	9295.332	5006.934
4	51+	60005	9463.662	5061.161

Next steps:

[Generate code with age_summary](#)

[New interactive sheet](#)

```

1 # Confidence Intervals - Age Life Stages
2
3 age_ci = {}
4
5 for age_group in walmart['Age_life_stage'].cat.categories:
6     purchase = walmart[walmart['Age_life_stage'] == age_group]['Purchase']
7
8     mean = purchase.mean()
9     std_error = purchase.std() / np.sqrt(len(purchase))
10
11     age_ci[age_group] = {}
12
13     for confidence in [0.90, 0.95, 0.99]:
14         age_ci[age_group][confidence] = stats.norm.interval(confidence=confidence, loc=mean, scale=std_error)

```

```

1 # Display Confidence Intervals
2
3 age_ci_summary = []
4
5 for confidence in [0.90, 0.95, 0.99]:
6     for age_group in walmart['Age_life_stage'].cat.categories:
7
8         lower = age_ci[age_group][confidence][0]
9         upper = age_ci[age_group][confidence][1]
10        mean = walmart[walmart['Age_life_stage'] == age_group]['Purchase'].mean()
11
12        age_ci_summary.append([
13            age_group,
14            f'{int(confidence * 100)}%',
15            mean,
16            lower,
17            upper
18        ])
19
20 age_ci_df = pd.DataFrame(
21     age_ci_summary,
22     columns=[
23         'Age Group',
24         'Confidence Level',
25         'Mean',
26         'Lower',
27         'Upper'
28     ]
29 )
30
31 # display(age_ci_df)
32 age_ci_table = pd.crosstab(
33     index=age_ci_df['Confidence Level'],
34     columns=age_ci_df['Age Group'],
35     values=age_ci_df['Mean'].round(3).astype(str) +
36         ' (' + age_ci_df['Lower'].round(2).astype(str) +
37         ', ' + age_ci_df['Upper'].round(2).astype(str) + ')',
38     aggfunc='first'
39 )
40
41 display(age_ci_table)

```

Age Group	0-17	18-25	26-35	36-50	51+
Confidence Level					
90%	8933.465 (8865.05, 9001.88)	9169.664 (9143.43, 9195.89)	9252.691 (9235.1, 9270.28)	9295.332 (9274.46, 9316.2)	9463.662 (9429.68, 9497.65)
95%	8933.465 (8851.95, 9014.98)	9169.664 (9138.41, 9200.92)	9252.691 (9231.73, 9273.65)	9295.332 (9270.46, 9320.2)	9463.662 (9423.17, 9504.16)

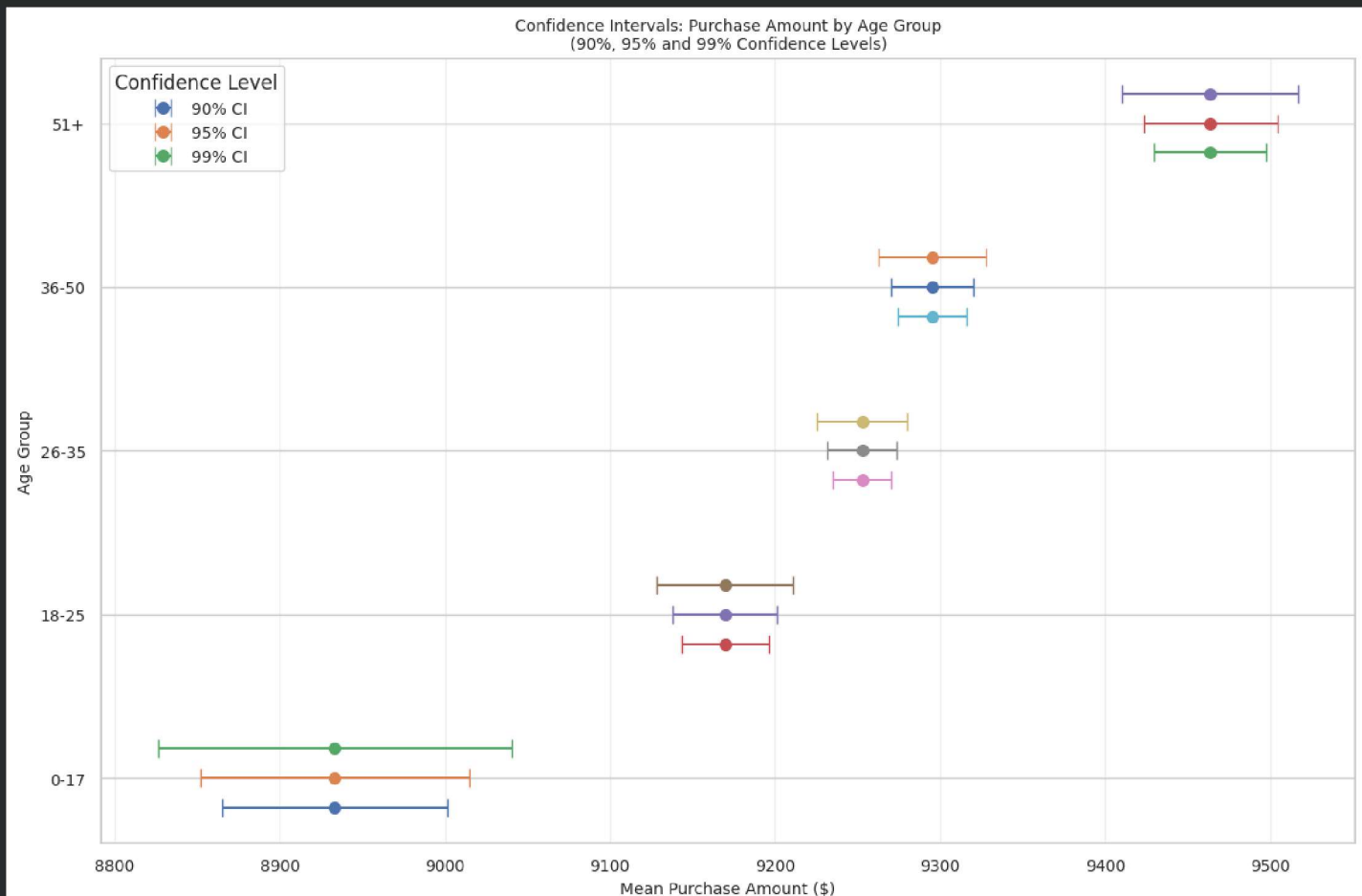
Next steps:

[Generate code with age_ci_table](#)[New interactive sheet](#)

```

1 # Confidence Intervals - Age Life Stages
2
3 plt.figure(figsize=(12, 8))
4
5 confidence_levels = [0.90, 0.95, 0.99]
6 age_groups = list(walmart['Age_life_stage'].cat.categories)
7
8 for i, age_group in enumerate(age_groups):
9     mean = walmart[walmart['Age_life_stage'] == age_group]['Purchase'].mean()
10
11     for j, confidence in enumerate(confidence_levels):
12
13         lower = age_ci[age_group][confidence][0]
14         upper = age_ci[age_group][confidence][1]
15
16         y_position = i + (j - 1) * 0.18
17
18         plt.errorbar(mean, y_position, xerr=[[mean - lower], [upper - mean]],
19                     fmt='o', capsize=6, markersize=7,
20                     label=f'{int(confidence * 100)}% CI' if i == 0 else None)
21
22 plt.yticks(
23     range(len(age_groups)),
24     age_groups
25 )
26
27 plt.xlabel('Mean Purchase Amount ($)')
28 plt.ylabel('Age Group')
29 plt.title(
30     'Confidence Intervals: Purchase Amount by Age Group\n'
31     '(90%, 95% and 99% Confidence Levels)'
32 )
33
34 plt.legend(title='Confidence Level')
35 plt.grid(axis='x', alpha=0.3)
36
37 plt.tight_layout()
38 plt.show()

```



- ▼ One-Way ANOVA Hypothesis Test across Age Life Stages, at different confidence levels
- ▼ H0: Null Hypothesis: There is no significance difference between male and female spending, at given confidence level
- Ha: There is a significant difference between male and female spendings

```
1 # significance levels corresponding to CI levels
2
3 alpha_levels = []
4 for ci in confidence_levels:
5     alpha_levels.append(round(1-ci, 2))
6 alpha_levels
```

```
[0.1, 0.05, 0.01]
```

```
1 from scipy.stats import f_oneway
```

```
1 anova_samples = [walmart[walmart['Age_life_stage'] == a]['Purchase'].values for a in age_groups]
2 for alpha in alpha_levels:
3     f_stat, p_value = f_oneway(*anova_samples)
4     if p_value < alpha:
5         print(f'P value for 2 sample T Test at {(1-alpha)*100}% confidence: {p_value}')
6         print(f'As p value < alpha, {alpha}, We reject Null Hypothesis, Mean purchase differ significantly across age li
7     else:
8         print(f"We fail to reject Null Hypothesis, Mean purchase differ significantly across age life stages.")
9
```

```
P value for 2 sample T Test at 90.0% confidence: 1.0584319453882012e-42
As p value < alpha, 0.1, We reject Null Hypothesis, Mean purchase differ significantly across age life stages.
P value for 2 sample T Test at 95.0% confidence: 1.0584319453882012e-42
As p value < alpha, 0.05, We reject Null Hypothesis, Mean purchase differ significantly across age life stages.
P value for 2 sample T Test at 99.0% confidence: 1.0584319453882012e-42
As p value < alpha, 0.01, We reject Null Hypothesis, Mean purchase differ significantly across age life stages.
```

Findings across Age Life Stages

1. Monotonic Spending Gradient:
- 0-17 : \$8,933.46 (Lowest spend per transaction; young dependents with limited budgets).

◦ 18-25 : \$9,169.66

◦ 26-35 : \$9,252.69 (Highest transaction volume: 219,587 transactions, 39.9% of total store activity).

◦ 36-50 : \$9,295.33

◦ 51+ : \$9,463.66 (Highest spend per transaction; established financial wealth and premium basket preference).
2. Statistical Significance: One-way ANOVA confirms significant differences ($F = 50.65, p = 1.06 \times 10^{-42}$).
3. Overlap Dynamics:
- The 51+ cohort's 95% CI [9, 423.17, 9,504.16] does NOT overlap with 26-35 , 18-25 , or 0-17 .

◦ The 0-17 cohort's 95% CI [8, 851.95, 9,014.98] is strictly detached below all other age brackets.

◦ The intermediate adult brackets (26-35 and 36-50) exhibit adjacent, slightly overlapping intervals.

8. Final Insights & Generalization to Population

Extrapolating to the 100 Million Customer Base (50M Male, 50M Female)

Assuming Walmart has 50 million male and 50 million female customers globally:

Metric	Female Population (50M)	Male Population (50M)	Total Projected Market
Point Estimate Spend / Transaction	\$8,734.57	\$9,437.53	\$9,263.97 (weighted)
95% Confidence Interval	[\$8,709.21, \$8,759.92]	[\$9,422.02, \$9,453.03]	Non-Overlapping
Average Transactions / Customer	~81.5 transactions	~98.0 transactions	~93.4 transactions
Annualized Total Customer Spend	~\$712,024	~\$925,344	~\$865,000
Top Volume Driver Category	Category 5 (Apparel/Home)	Category 1 (Electronics/Appliances)	Category 1 & 5
Geographic Epicenter	City B & City C	City B & City C	City B (Tier-2 Powerhouse)

- **The True Differentiator:** It is **not** marital status, and **not** city residency duration. The two primary drivers of Black Friday basket revenue are:
1. **Gender-Driven Category Mix Shift:** High-ticket Category 1 vs standard retail goods.

9. Actionable Business Recommendations for Walmart Management

1. Bundle High-Margin Electronics with Lifestyle Products for Female Shoppers

- **Action:** Women spend the same dollar amount as men when buying Category 1 items, but buy them less frequently. Create curated Black Friday bundles—e.g. smart home assistants, premium headphones, or fitness trackers packaged with popular apparel or home goods at an attractive package discount.
- **Goal:** Elevate female transaction size toward the \$9,400+ mark.

2. Safeguard High-Ticket Inventory (Product Category 1) for Peak Male Shoppers

- **Action:** Men account for 75.3% of transactions and 27.9% of their purchases are in high-ticket Category 1.
- **Goal:** Implement dynamic inventory pre-allocation to prevent out-of-stock events on top-selling SKUs in Category 1, particularly in City B stores.

3. VIP Fast-Track & Premium Curation for Senior Shoppers (Ages 51+)

- **Action:** Shoppers aged 51+ have the highest average spend per purchase (\$9,463.66). Offer early-access digital shopping hours, premium home delivery, and dedicated concierge customer service during Black Friday.
- **Goal:** Maximize high-margin, high-ticket conversions for your most affluent customer cohort.

4. Eliminate Ineffective Marital-Status Segmentation

- **Action:** Married and unmarried customers spend virtually identical amounts (\$9,261 vs \$9,265).
- **Goal:** Consolidate marketing campaigns and stop allocating creative or media spend toward separate single vs married promotions. Reallocate those marketing dollars toward category-specific campaigns.

5. Expand Fulfillment & Local Merchandising in City Category B

- **Action:** City B generates 42% of total transaction volume and exhibits consistent high average transaction sizes.
- **Goal:** Expand regional fulfillment micro-hubs and prioritize inventory replenishment runs to City B distribution centers ahead of the holiday rush.

End of Report — Walmart Customer Purchase Behavior Analysis

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.

1 Start coding or generate with AI.